

RAMA: INFORMÁTICA	CICLO: DESENVOLVIMIENTO DE APLICACIONES MULTIPLATAFORMA
MÓDULO: ACCESO A DATOS	CURSO: 2º
PROFESOR: IAGO DOVAL IGLESIAS	
AVALIACIÓN: RECUPERACIÓN – MODELO D	GRUPO: ÚNICO DATA: Junio 2026
NOMBRE ALUMNO:	DNI:

El examen consta de **tres partes**. Nota mínima total: **5 puntos**, con al menos **1 punto** en cada parte. Entrega un único comprimido: **1erApellido_2oApellido_Nombre**. Trata todas las excepciones y libera todos los recursos (-0.10 pt por elemento no tratado). Muestra un ejemplo de invocación por ejercicio (-0.15 pt si falta).

PARTE 1 – Ficheros (3 pt)

1) **Ficheros (1.30 pt)**. Crea el método **aTexto** que transforme un fichero binario de objetos *Libro* a un fichero de texto. La primera línea del fichero binario es una cabecera con los nombres de los campos. Cada libro ocupa una línea en el fichero de texto. Los campos son: **año de publicación** (entero), **valoración** (float), **título** (cadena), **autor** (cadena). Si el fichero origen no existe o está vacío, el fichero destino quedará vacío. El nombre del fichero de texto será el mismo que el binario pero con extensión .txt. Como datos de prueba puedes usar **libros.dat**.

2) **Ficheros (1.70 pt)**. Crea el método **transform** que transforme un fichero binario de tipos primitivos a un fichero de texto. El binario contiene secuencias *Int String Int String ...* donde el entero indica cuántas veces se escribe la cadena en el fichero de texto. Como datos de prueba puedes usar **transform.dat**.

PARTE 2 – DOM / SAX (3.50 pt)

El fichero **carreras.xml** contiene las 2 primeras carreras de F1 de 2019. Toda la información mostrada debe extraerse de este fichero.

3) **DOM (0.60 pt)**. Muestra la temporada (season) y el nombre de cada circuito (name dentro de Circuit).

4) **DOM (0.90 pt)**. Muestra, para cada carrera (Race), el país (country), la longitud (long) y latitud (lat) y el nombre de cada escudería (name dentro de Constructor). **Sin repetir ninguna escudería**.

5) **DOM (1.00 pt)**. De cada carrera, muestra el nombre de la carrera (raceName), el código del circuito (atributo code de Circuit) y el nombre del piloto (name dentro de Driver) que quedó en **tercera posición** (position = 3).

6) **SAX (1.00 pt)**. Muestra, para cada carrera (Race), el país (country), la longitud (long) y latitud (lat), el código del circuito (atributo code de Circuit) y el nombre de cada escudería (name dentro de Constructor). **Sin repetir ninguna escudería**. La temporada (season) se mostrará solo una vez, al principio.

PARTE 3 – Servicios Web (3.50 pt)

Condiciones generales:

- El path del servicio web será **juego**. (-0.10 pt si se especifica otro).
- Si no se especifica path en el ejercicio, se usará el del punto anterior.
- Todos los métodos deben devolver un objeto tipo **Response**. En caso de error, devuelve código de error y mensaje explicativo.
- Si no se especifica lo contrario, se produce y/o consume **JSON y XML**. Para listas, solo JSON. (-0.10 pt si se especifica otro).
- Crea un **ArrayList estático** llamado **misJuegos** para los ejercicios 7 y 8. La BD solo se usa en los ejercicios 9 y 10.

7) **(1.00 pt)** Un método que cree un juego con valores fijos y lo añada al ArrayList **misJuegos**. No se pasa nada en el Body. No hay que controlar códigos repetidos. Devuelve el juego creado.

- 8) **(1.20 pt)** Un método en el path "{id}" que borre del ArrayList **misJuegos** todos los juegos cuyo código sea igual a *id*. Devuelve el número de juegos borrados.
- 9) **(0.70 pt)** Un método que devuelva todos los juegos de la base de datos en los que el *precioalquiler* sea nulo.
- 10) **(0.60 pt)** Un método que permita actualizar, si existe en la base de datos, el nombre, el stock de venta y el precio de venta de un juego identificado por su id. El id se pasa como **QueryParam** y los nuevos datos como un objeto **JSON** en el Body. Se debe avisar si el juego no existe.